Práctica 01: tiempo de ejecución con flujos con y sin buffer en Java

Oct 17, 2026 · @Rodrigo

Los flujos con buffer fueron más rápidos en los 12 pares comparados: entre 1.7 y 2.6 veces en texto y entre 44 y 119 veces en binario. En binario, la diferencia crece con el tamaño de los datos.

1. Objetivo

Generar y leer archivos de texto y binarios con `FileWriter`, `BufferedWriter`, `FileReader`, `BufferedReader`, `FileOutputStream`, `BufferedOutputStream`, `FileInputStream` y `BufferedInputStream`, y medir el tiempo de cada flujo con `System.nanoTime()`. Materia: Concurrencia y Paralelismo.

2. Entorno y método

- **Programa:** `EscrituraBuffered.java` (archivo adjunto), con los ocho flujos en `try-with-resources`. Cada medición incluye el vaciado y cierre final del flujo.
- **Elementos:** en texto, cada elemento es un número entero con salto de línea; en binario, cada elemento es un byte (`write(i)` / `read()`).
- **Calentamiento:** antes de medir, el programa ejecuta los ocho flujos con 10,000 elementos y descarta ese resultado (JIT de la JVM).
- **Repeticiones:** 3 ejecuciones independientes de la JVM por cada valor de n (100,000, 1,000,000 y 10,000,000).
- **Equipo de medición:** Windows, Java JDK 23, ejecutado desde la terminal de VS Code. Los tiempos absolutos dependen del equipo; lo que se compara es el comportamiento de los flujos entre sí.

3. Resultados

Tiempos en milisegundos. El promedio es de las tres ejecuciones.

n = 100,000

Flujo utilizado	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	25.8	17.2	16.9	20.0

Flujo utilizado	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
BufferedWriter	10.6	11.2	12.5	11.4
FileReader	17.0	17.2	17.9	17.3
BufferedReader	10.4	9.8	9.7	10.0
FileOutputStream	168.2	172.8	167.7	169.5
BufferedOutputStream	3.0	4.4	3.8	3.7
FileInputStream	120.5	128.2	120.4	123.0
BufferedInputStream	2.9	2.7	2.9	2.8

n = 1,000,000

Flujo utilizado	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	119.2	101.9	77.7	99.6
BufferedWriter	37.2	39.2	36.8	37.7
FileReader	209.7	211.5	190.2	203.8
BufferedReader	100.6	94.0	92.3	95.6
FileOutputStream	1676.5	1724.1	1603.7	1668.1
BufferedOutputStream	16.1	17.4	16.5	16.7
FileInputStream	1176.2	1292.0	1168.5	1212.2
BufferedInputStream	16.6	17.5	15.3	16.4

n = 10,000,000

Flujo utilizado	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
FileWriter	685.2	657.4	653.6	665.4
BufferedWriter	281.0	275.2	269.2	275.1
FileReader	2171.7	2116.9	2119.4	2136.0
BufferedReader	1041.5	1030.3	1036.8	1036.2
FileOutputStream	16248.5	16196.4	16271.1	16238.7

Flujo utilizado	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
BufferedOutputStream	139.1	135.8	135.4	136.8
FileInputStream	11478.3	11390.8	11468.5	11445.9
BufferedInputStream	130.7	131.4	133.7	131.9

Cuántas veces más rápido es el flujo con buffer

Cociente promedio sin buffer / con buffer.

Comparación	n = 100,000	n = 1,000,000	n = 10,000,000
Texto, escritura (FileWriter / BufferedWriter)	1.7×	2.6×	2.4×
Texto, lectura (FileReader / BufferedReader)	1.7×	2.1×	2.1×
Binario, escritura (FileOutputStream / BufferedOutputStream)	45.6×	100.0×	118.7×
Binario, lectura (FileInputStream / BufferedInputStream)	43.6×	73.8×	86.7×

La primera ejecución de `FileWriter` con n = 100,000 (25.8 ms) fue más lenta que las otras dos (cerca de 17 ms). En el resto de las mediciones los tres valores de cada flujo son parecidos entre sí.

4. Análisis

1. Diferencia entre flujos con buffer y sin buffer. Los flujos con buffer fueron más rápidos en todos los casos. La causa es que cada llamada a `FileOutputStream.write(int)` o `FileInputStream.read()` es una llamada al sistema operativo (cambio de modo usuario a núcleo) por un solo byte. El buffer acumula 8 KB en memoria y hace una sola llamada por bloque, así que reduce esas llamadas unas 8,000 veces. Con 10,000,000 de bytes, la escritura bajó de 16238.7 ms a 136.8 ms, unas 119 veces menos.

2. Variación del tiempo al aumentar n. El tiempo crece con n. En los flujos de bytes sin buffer el crecimiento es casi lineal: `FileOutputStream` pasó de 169.5 ms a 1668.1 ms y a 16238.7 ms (9.8 y 9.7 veces por cada aumento de 10 veces en n), unos 1.6 µs por byte. Con buffer, el crecimiento es menor que lineal en los valores pequeños: `BufferedOutputStream` pasó de 3.7 ms a 16.7 ms (4.5 veces) y luego a 136.8 ms (8.2 veces), porque con pocos datos pesan los costos fijos (abrir el archivo, el JIT). En texto, la

lectura creció un poco más que linealmente (`FileReader` : 11.8 y 10.5 veces), porque el archivo de texto crece más rápido que n: cada número tiene más dígitos (0.6 MB, 6.9 MB y 78.9 MB). Escritura y lectura siguen el mismo patrón general.

3. Caracteres contra bytes. Con buffer, el binario fue más rápido por elemento: `BufferedOutputStream` tardó 136.8 ms y `BufferedWriter` 275.1 ms para n = 10,000,000, porque el texto necesita convertir cada número a cadena y codificar los caracteres según el *charset*. Sin buffer ocurrió lo contrario: `FileWriter` (665.4 ms) fue mucho más rápido que `FileOutputStream` (16238.7 ms). Esto se debe a que `FileWriter` ya trae un buffer interno de 8 KB en su codificador, mientras que `FileOutputStream` no tiene ninguno. Lo mismo pasa en lectura: `FileReader` leyó unos 79 millones de caracteres en 2136.0 ms, y `FileInputStream` solo 10 millones de bytes en 11445.9 ms. Por eso la mejora de `BufferedWriter` (2.4×) es mucho menor que la de `BufferedOutputStream` (119×). Además, los archivos de texto son más grandes: con n = 10,000,000 el texto ocupa unos 79 MB y el binario 10 MB.

4. Tamaño del buffer por defecto. `BufferedOutputStream` y `BufferedInputStream` usan 8192 bytes (8 KB). `BufferedWriter` y `BufferedReader` usan 8192 caracteres (16 KB en memoria, porque cada `char` ocupa 2 bytes). Sí se puede cambiar con el segundo argumento del constructor, por ejemplo `new BufferedOutputStream(fos, 65536)` .

5. `readLine()` en `BufferedReader` y no en `BufferedInputStream` . `BufferedReader` trabaja con caracteres ya decodificados, así que puede buscar el salto de línea (`\n` o `\r\n`) y devolver la línea como `String` . `BufferedInputStream` trabaja con bytes sin interpretación y no sabe qué codificación tienen ni qué es una línea. Al leer texto con flujos de bytes habría que buscar el byte `10` a mano y decodificar después, con el riesgo de cortar un carácter de varios bytes (UTF-8) a la mitad. Por eso, para texto se usa `BufferedReader` .

5. Conclusión

El buffer reduce el número de accesos al sistema de archivos y, con ello, el tiempo de ejecución. La mejora es enorme en flujos de bytes (de 44 a 119 veces) y moderada en texto (de 1.7 a 2.6 veces), porque `FileWriter` y `FileReader` ya incluyen un buffer interno. Por eso, al leer o escribir byte por byte o carácter por carácter, conviene usar siempre `BufferedOutputStream` , `BufferedInputStream` , `BufferedWriter` o `BufferedReader` dentro de un `try-with-resources` , que además garantiza el vaciado y cierre del flujo. Estas mediciones son simples; la JVM aplica optimizaciones y, para resultados más rigurosos, habría que usar JMH y repetir más corridas.